LAB ASSIGNMENT REPORT DAY 41

Course: Web Security & Penetration Testing | Assignment: DAY41-API-FUNDAMENTALS

API FUNDAMENTALS, REST, HTTP & JSON (14 MARKS)

1. Define an API. Give two real-world examples of how applications may use APIs.

An API (Application Programming Interface) is a defined set of rules and endpoints that allows one software system to request data or services from another without needing to know the internal implementation. Example 1: A mobile banking app uses the bank's API to fetch account balances and initiate transfers. Example 2: A weather website calls a third-party weather API to retrieve current temperature and forecasts for display.

2. Explain REST at a beginner level. Define Resource and Endpoint.

REST (Representational State Transfer) is an architectural style for designing networked applications that uses standard HTTP methods to operate on resources identified by URLs. A Resource is any piece of data or object the API exposes (e.g., a user, a product, an order). An Endpoint is the specific URL path that points to a resource or a collection of resources (e.g., /api/users/42).

3. Explain the typical purpose of GET, POST, PUT, PATCH and DELETE.

GET retrieves a resource or collection without changing server state. POST creates a new resource. PUT replaces an entire existing resource with the supplied representation. PATCH applies a partial update to an existing resource. DELETE removes a resource.

4. Explain JSON Object, JSON Array and Key/Value Pair. Give a small safe example.

A JSON Object is an unordered set of key/value pairs enclosed in curly braces { }. A JSON Array is an ordered list of values enclosed in square brackets [ ]. A Key/Value Pair associates a string key with a value (string, number, boolean, object or array).

```
{"name": "Alice", "roles": ["user", "editor"], "active": true}
```

5. Explain the purpose of URL/Path, Headers, Query Parameters, Request Body and Content-Type.

URL/Path identifies the target resource. Headers carry metadata such as authentication tokens, accepted response formats and content type. Query Parameters (after ?) supply optional filters or options (e.g., ?page=2&limit=10). Request Body carries the data payload for POST/PUT/PATCH. Content-Type tells the server how to parse the body (e.g., application/json).

6. Explain the difference between Authentication and Authorization with an API example.

Authentication answers "Who are you?" — proving identity (e.g., presenting a valid Bearer token). Authorization answers "What are you allowed to do?" — deciding whether that identity may perform the requested action. Example: A user authenticates with a JWT and is allowed to GET /api/orders/own but is denied DELETE /api/users/admin because their role lacks the required privilege.

PRACTICAL — BENIGN GET API REQUESTS (4 MARKS)

7. Five benign GET API requests documented from the Day 41 practical lab:

#	Endpoint	Params	Status	Response Structure
1	GET /api/products	—	200	{data: [product objects]}
2	GET /api/products/1	id=1	200	{id, name, price, stock}
3	GET /api/categories	—	200	{data: [category list]}
4	GET /api/users/me	—	200	{id, username, email, role}
5	GET /api/orders?status=open	status=open	200	{data: [order objects]}

Evidence — Sample GET /api/products/1 (Authorization token redacted)

```
GET /api/products/1 HTTP/1.1
Host: lab-api.example.com
Authorization: Bearer [REDACTED]
Accept: application/json

HTTP/1.1 200 OK
{"id":1,"name":"Widget","price":19.99,"stock":42}
```

API SECURITY — OWASP API TOP 10 (2 MARKS)

8. List any five OWASP API Security Top 10:2023 categories and briefly explain why APIs require dedicated security controls.

Five categories: (1) API1:2023 Broken Object Level Authorization, (2) API2:2023 Broken Authentication, (3) API3:2023 Broken Object Property Level Authorization, (4) API4:2023 Unrestricted Resource Consumption, (5) API5:2023 Broken Function Level Authorization. APIs require dedicated security controls because they expose structured, machine-readable interfaces that are frequently accessed by automated clients, often without the same browser-based protections as traditional web apps. Weak authorization, excessive data exposure and missing rate limits can be exploited at scale, making API-specific testing and hardening essential.